see the project you've added and not to see the project you haven't. You don't need to cover all the possibilities of admins and public projects, for the same reason as before. From the controller's perspective it's all the same call, and you just need to test that call is made.

This test fails on the last line because you haven't actually implemented the restrictions yet. You do have to be careful with not_to have_selector—it's easy for a typo to look like a passing test.

On the user side, this functionality must deal with admin users and public projects. The logic is parallel to the can_view? logic.

Here are the unit tests for the User behavior of being able to access a list of projects. Again, I wrote them and passed them one at a time:

security/04/spec/models/user_spec.rb

```
describe "visible projects" do
  let!(:project_1) { create(:project, name: "Project 1") }
  let!(:project_2) { create(:project, name: "Project 2") }

  it "allows a user to see their projects" do
    user.projects << project_1
    expect(user.visible_projects).to eq([project_1])
  end

  it "allows an admin to see all projects" do
    user.admin = true
    expect(user.visible_projects).to match_array(Project.all)
  end

  it "allows a user to see public projects" do
    user.projects << project_1
    project_2.update(public: true)
    expect(user.visible_projects).to match_array([project_1, project_2])
  end

  it "has no duplicates in project list" do
    user.projects << project_1
    project_1.update(public: true)
    expect(user.visible_projects).to match_array([project_1])
  end
end
```

A few notes on these unit tests: The user is created by a let earlier in the file. The projects need to be created using let! rather than let. This is a common problem when you're testing ActiveRecord database access for objects that you don't otherwise refer to. If you don't refer to them before trying to find them, then let's lazy initialization doesn't create them, so you need to force creation with let!.